

ViewModelBase 생성자 매개변수

● ViewModelBase 상속시 생성자에서 매개변수 상용방법

```
public class ViewModelA : ViewModelBase
{
    private readonly Lazy<ViewModelB> _viewModelB;

    public ViewModelB OtherViewModel => _viewModelB.Value;

    public ViewModelA(IContainerProvider container, ILoggerService logger)
        : base(container, logger)
    {
        _viewModelB = new Lazy<ViewModelB>(() => Container.Resolve<ViewModelB>());
    }
}
```

Lazy<T>를 사용하여 ViewModelB를 지연 로딩(lazy loading)하는 이 방식은 상당히 좋은 방식입니다.

특히 다음과 같은 경우에 의도적으로 사용할 만한 전략적 설계입니다:

● 이 방법이 좋은 이유

이유	설명
지연 로딩 (Lazy Loading)	ViewModelB가 실제로 필요해질 때까지 생성되지 않으므로 불필요한 메모리/초기화 비용을 절감합니다.
순환 참조 방지	ViewModelA → ViewModelB, 그리고 반대로 ViewModelB → ViewModelA 구조가 있을 때 DI 순환 종속성 문제를 피할 수 있습니다.
느슨한 결합 (Loose Coupling)	Container.Resolve<T>()를 통해 명시적 종속성을 늦추면 구조가 더 유연해지고, 향후 리팩터링에 유리합니다.
테스트 유연성	테스트 시 Lazy<T> 안의 Resolve 코드를 모킹하거나, Container만 주입해도 되므로 단위 테스트 작성이 쉬워집니다.

● 주의할 점

주의사항	설명
예외 발생 시점	ViewModelB가 실제로 사용되는 시점에서 Resolve 에 실패하면 예외가 런타임 중 늦게 발생합니다. 초기화 시점이 아니므로 디버깅이 다소 어려울 수 있습니다.

주의사항	설명
컨테이너 의존 증가	DI 컨테이너에 너무 많은 책임을 맡기면 서비스 로케이터 패턴처럼 설계가 흐려질 수 있으므로 남용은 피해야 합니다.
단위 테스트 시 조심	테스트할 때 Container.Resolve 를 모킹하거나 대체 구현을 명시적으로 주입해줘야 합니다. (MoQ, Fake Container 등 필요할 수 있음)

● 추천사항

- ViewModel 간 강한 연결을 피하고 싶을 때
- 특정 ViewModel이 일부 조건에서만 사용될 때
- 성능 최적화가 필요한 복잡한 UI 구조일 때
- 종속성 수가 많고 구성 복잡도가 높을 때

● 그렇다면 언제 일반 생성자 주입을 써야 하나요?

- ViewModelB가 항상 필요하고, ViewModelA가 생성될 때 반드시 존재해야 할 때
- 순환 종속성 문제가 없고 구조가 단순할 때
- 테스트 시 명확한 명세가 필요한 경우

● 결론

```
public class ViewModelA : ViewModelBase
{
    private readonly Lazy<ViewModelB> _viewModelB;
    public ViewModelB OtherViewModel => _viewModelB.Value;
    public ViewModelA(IContainerProvider container, ILoggerService logger)
        : base(container, logger)
    {
        _viewModelB = new Lazy<ViewModelB>(() => Container.Resolve<ViewModelB>());
    }
}
```

이 방식은 ->

- 성능을 고려한 설계
- 유연성 있는 아키텍처
- 테스트 가능성이 보장됨

→ 전문적인 ViewModel 설계로도 매우 적합한 방식입니다.

● _viewModelB.Value; 설명

C#의 Lazy<T> 타입에서 내부 개체를 실제로 가져오는 속성입니다.
핵심적으로는 게으른(lazy) 초기화를 수행합니다

● 기본 구조

```
private readonly Lazy<ViewModelB> _viewModelB;  
public ViewModelB OtherViewModel => _viewModelB.Value;
```

여기서 Lazy<ViewModelB>는 ViewModelB 개체를 나중에,
즉 처음 사용할 때만 생성되도록 지연시킵니다

● 어떻게 동작하나요

```
_viewModelB = new Lazy<ViewModelB>(() => Container.Resolve<ViewModelB>());
```

- 이 코드는 ViewModelB를 생성하기 위한 팩토리 메서드(delegate) 를 넘깁니다.
- 실제로 ViewModelB 개체는 생성되지 않습니다(아직)

```
var b = _viewModelB.Value;
```

- 처음 이 라인을 실행하면, 내부적으로 new ViewModelB(...)처럼 생성자가 실행됩니다.
- 그 후에는 같은 인스턴스를 재사용합니다.
즉, 내부적으로 캐시됨 : Lazy<T>는 **한 번만 생성**합니다.

● 예제로 이해하기

```
public class ViewModelA  
{  
    private readonly Lazy<ViewModelB> _viewModelB;  
    public ViewModelA()  
    {  
        Console.WriteLine("ViewModelA 생성자 실행됨");  
        _viewModelB = new Lazy<ViewModelB>(() =>  
        {  
            Console.WriteLine("ViewModelB 생성됨!");  
            return new ViewModelB();  
        });  
    }  
}
```

```
public void UseB()
{
    Console.WriteLine("B를 사용하기 직전...");
    var b = _viewModelB.Value; // 이 순간 ViewModelB가 생성됨
    Console.WriteLine("B를 사용했습니다.");
}
}
```

출력은 다음과 같습니다:

```
ViewModelA 생성자 실행됨
B를 사용하기 직전...
ViewModelB 생성됨!
B를 사용했습니다.
```

_viewModelB.Value는 처음 접근할 때만 ViewModelB를 생성합니다

● 요약

항목	설명
.Value	내부 객체를 가져오는 속성 (필요 시 생성됨)
처음 한 번만	내부 Func<T>가 딱 한 번 실행됨
이후에는 캐시	두 번째부터는 같은 객체 반환
성능 최적화	무거운 객체를 필요할 때까지 생성하지 않음
안전함	Lazy<T>는 스레드 안전(Thread-safe) 옵션도 제공 가능

● 언제 유용한가

- ViewModel에서 무거운 작업을 지연하고 싶을 때
- 순환 종속성을 피하고 싶을 때
- 실제로 사용되지 않을 수도 있는 객체를 생성하지 않도록
- 성능을 세밀하게 제어하고 싶을 때

Lazy<T>를 비즈니스 로직에서 데이터베이스 입력에 활용할 수 있으나, 트랜잭션과 상태 관리를 꼼꼼히 고려해야 하며, 잘못 사용하면 의도치 않은 타이밍 문제나 상태 공유 문제 발생 가능하므로 주의가 필요합니다.

● Repository<T>와 ViewModel에서 Lazy 주입 결합 예시

```

public class UserViewModel : ViewModelBase
{
    private readonly Lazy<Repository<User>> _userRepository;

    public UserViewModel(IContainerProvider container)
        : base(container)
    {
        // Repository<User>는 생성 비용이 비교적 크거나,
        // 실제로 필요할 때만 생성하길 원할 때 Lazy 사용
        _userRepository = new Lazy<Repository<User>>(() =>
            Container.Resolve<Repository<User>>());
    }

    public async Task SaveUserAsync(User user)
    {
        if (user == null) throw new ArgumentNullException(nameof(user));
        // 실제 DB 저장 시점에 Repository<User> 인스턴스 생성
        await _userRepository.Value.CreateAsync(user);
    }
}

```

● 왜 이렇게 하는 게 좋은가?

● 지연 초기화 (Lazy Initialization)

UserViewModel 인스턴스가 생성될 때 바로 Repository<User>를 만들지 않고, SaveUserAsync 호출 시점까지 미뤄서 초기화 비용 절감.

● DI 컨테이너와 결합

Prism이나 다른 DI 컨테이너가 관리하는 Repository<User>를 직접 Resolve하기 때문에, 종속성 관리가 깔끔해짐.

● 테스트 용이성

테스트할 때 DI 컨테이너 설정만 맞추면 쉽게 Mock Repository 주입 가능.

● 주의할 점

● Lazy<T> 내부에서 DI 컨테이너를 호출하므로, 컨테이너가 Repository<T>를 정확히 등록해야 함.

● 트랜잭션 관리가 중요한 경우, Repository<T> 내부 메서드 호출 시점과 트랜잭션 경계가 일치하도록 설계 필요.

● 멀티스레드 환경이라면 기본 Lazy<T>는 스레드 안전하므로 크게 문제 없지만,

상황에 따라 LazyThreadSafetyMode 조절 가능.

● 설계 확장 팁

- Repository<T> 인터페이스(IRepository<T>)를 만들어서 ViewModel은 인터페이스에 종속하도록 하면 더 유연함.
- 필요하다면 ViewModelBase에 기본적인 Lazy<T> Resolve 헬퍼 메서드를 만들어 중복 코드를 줄일 수 있음. (아래 참고)

● 요약

장점	설명
초기화 비용 절감	실제 필요한 시점까지 생성 지연
DI와 결합 용이	Prism 등 DI 컨테이너에서 Resolve해 의존성 주입 관리
테스트 친화적	인터페이스 활용, Mock 주입 등 테스트 용이
순환 참조 문제 해결	Lazy를 활용해 의존성 순환 문제 완화

● Prism DI 컨테이너에 제네릭 Repository 등록하기

보통 Prism 앱에서는 RegisterTypes 메서드에서 DI 등록을 합니다.

```
protected override void RegisterTypes(IContainerRegistry containerRegistry)
{
    // DbContext는 보통 싱글톤 또는 스코프 단위로 등록
    containerRegistry.RegisterSingleton<AppDbContext>();

    // Repository<T>는 Transient(매번 새로 생성)로 등록
    containerRegistry.Register(typeof(IRepository<>), typeof(Repository<>));

    // 그 외 ViewModel, 서비스 등 등록
    containerRegistry.Register<ViewModelA>();
    containerRegistry.Register<ViewModelB>();
}
```

Register(typeof(IRepository<>), typeof(Repository<>))
제네릭 타입을 Prism DI 컨테이너에 등록하는 구문입니다.

이로써 IRepository<User>를 요청하면 Repository<User> 인스턴스를 받을 수 있습니다.

AppDbContext 같은 DbContext는 보통 싱글톤이나 Scoped로 등록합니다.
(앱 상황에 따라)

● ViewModel에서 Lazy 서비스 활용 예시

```

public class UserViewModel : ViewModelBase
{
    private readonly Lazy<IRepository<User>> _userRepository;

    public UserViewModel(IContainerProvider container)
        : base(container)
    {
        // Lazy로 DI 컨테이너에서 IRepository<User> Resolve
        _userRepository = new Lazy<IRepository<User>>(() =>
            Container.Resolve<IRepository<User>>());
    }

    public async Task SaveUserAsync(User user)
    {
        if (user == null) throw new ArgumentNullException(nameof(user));
        // 실제 DB 작업 시점에 Repository 인스턴스가 생성 및 반환됨
        await _userRepository.Value.CreateAsync(user);
    }
}

```

Lazy<IRepository<User>>를 통해 Repository가 실제 필요한 순간에만 생성됩니다.

● 여러 서비스 함께 Lazy로 사용하는 패턴

```

public class ComplexViewModel : ViewModelBase
{
    private readonly Lazy<IRepository<Order>> _orderRepository;
    private readonly Lazy<ILoggerService> _logger;

    public ComplexViewModel(IContainerProvider container)
        : base(container)
    {
        _orderRepository = new Lazy<IRepository<Order>>(() =>
            Container.Resolve<IRepository<Order>>());

        _logger = new Lazy<ILoggerService>(() => Container.Resolve<ILoggerService>());
    }

    public async Task ProcessOrderAsync(Order order)
    {
        _logger.Value.Log("Processing order...");
        await _orderRepository.Value.CreateAsync(order);
    }
}

```

```

        _logger.Value.Log("Order processed.");
    }
}

```

여러 서비스를 Lazy로 선언해 필요할 때만 초기화하는 패턴.

● ViewModelBase에서 Lazy 헬퍼 메서드 만들기 (옵션)

중복 코드 줄이기 위해 ViewModelBase에 헬퍼 메서드 추가 가능:

```

protected Lazy<T> ResolveLazy<T>() where T : class
{
    return new Lazy<T>(() => Container.Resolve<T>());
}

```

사용방법

```

_userRepository = ResolveLazy<IRepository<User>>();

```

● 요약

단계	설명
1	IRepository<T>를 Prism DI 컨테이너에 제네릭으로 등록
2	ViewModel 생성자에서 Lazy<IRepository<T>> 형태로 DI Resolve
3	실제 서비스 사용 시점에 Lazy.Value로 인스턴스 초기화 및 호출
4	필요하면 ViewModelBase에서 헬퍼 메서드로 중복 최소화

● ViewModelBase 에 ResolveLazy<T> 추가시 사용 예제

```

public class UserViewModel : ViewModelBase
{
    private readonly Lazy<IRepository<User>> _userRepository;

    public UserViewModel(IContainerProvider container) : base(container)
    {
        _userRepository = ResolveLazy<IRepository<User>>(); // 간단히 표기
    }

    public async Task SaveUserAsync(User user)
    {
        await _userRepository.Value.CreateAsync(user);
    }
}

```

● ViewModelBase 상속받은 UserViewModel 과 User 모델클래스 구현 예제

● 모델

```
public class Person
{
    public int Id { get; set; }
    public string? Name { get; set; }
    public int Age { get; set; }
    public DateTime BirthDay { get; set; }
    public string? Address { get; set; }
}
```

● UserViewModel

```
public class MainViewModel : ViewModelBase
{
    #region 필수 속성 패턴
    private bool _isMenuOpen;
    public bool IsMenuOpen
    {
        get => _isMenuOpen;
        set => SetProperty(ref _isMenuOpen, value);
    }
    private string? _username;
    public string? Username
    {
        get => _username;
        set => SetProperty(ref _username, value);
    }
    #endregion

    #region 필수 Commands
    private DelegateCommand? _loginCommand;
    public DelegateCommand? LoginCommand =>
        _loginCommand ??= new DelegateCommand(ExecuteLogin);

    private DelegateCommand? _informationCommand;
    public DelegateCommand? InformationCommand =>
        _informationCommand ??= new DelegateCommand(ExecuteOepnInformationView);
    private void ExecuteLogin()
    {
    }
    private void ExecuteOepnInformationView()
    {
    }
    #endregion
}
```

```

{
    Username = "로그인시 사용자이름표기"; // 로그인 후 사용자 이름 설정
}

private void ExecuteOepnInformationView()
{
    // 팝업을 새로 띄움
    InformationWindow window = new InformationWindow();

    // 현재 활성 창이 있다면 Owner 설정
    window.Owner = Application.Current.Windows
        .OfType<InformationWindow>()
        .FirstOrDefault(w => w.IsActive);

    window.ShowDialog(); // 또는 Show()로 비모달
}
#endregion

public ObservableCollection<Person> People { get; } = new();

private DelegateCommand? _loadUsersCommand;
public DelegateCommand LoadUsersCommand =>
    _loadUsersCommand ??= new DelegateCommand(async () =>
        await LoadUsersAsync(), () => !IsBusy).ObservesProperty(() => IsBusy);

public MainViewModel(IContainerProvider container) : base(container)
{
    Title = "사용자 목록";
}

private async Task LoadUsersAsync()
{
    // 기본작업 취소
    CancelCurrentTask();

    IsBusy = true;

    BusyMessage = "사용자 목록을 불러오는 중...";
}

```

```

try
{
    // 실제 API 호출이나 데이터베이스 쿼리 등 비동기 작업을 시뮬레이션합니다.
    await Task.Delay(2000, CancellationToken);

    // 예시 데이터 생성
    var dummyUsers = new List<Person>
    {
        new Person { Id = 1, Name = "홍길동", Age = 25, BirthDay = DateTime.Now, Address = "조선" },
        new Person { Id = 2, Name = "김철수", Age = 45, BirthDay = DateTime.Now, Address = "한국" }
    };

    await RunOnUiThread(async () =>           // ViewModelBase 에서
    {
        // UI 스레드에서 ObservableCollection에 데이터 추가
        People.Clear();
        foreach (var user in dummyUsers)
        {
            People.Add(user);
        }

        // UI 스레드에서 비동기 작업을 실행하기 위해 await 사용
        await Task.CompletedTask;
    });

    // BusyMessage 완료메세지 설정
    BusyMessage = "사용자 목록 불러오기가 완료되었습니다.";
}
catch (OperationCanceledException)
{
    BusyMessage = "사용자 목록 불러오기가 취소되었습니다.";
}
finally
{
    {
        IsBusy = false;
    }
}
}

```

이것은 텍스트포 사용자 목록 불러오기

<Grid Grid.Row="11" Margin="0 10 0 0">

```

<StackPanel Margin="10">
    <TextBlock Text="{Binding Title}" FontSize="18" FontWeight="Bold"
        Margin="0 0 0 10" />
    <Button Content="사용자 불러오기"
        Command="{Binding LoadUsersCommand}"
        IsEnabled="{Binding IsBusy, Converter={StaticResource InverseBooleanConverter}}" />

    <!-- IsEnabled="{Binding IsBusy}" 삭제 -->
    <!-- ViewModel과 View 연결 (ViewModelLocator 또는 Region 등록 시) -->
    <ctrls:LoadingOverlay
        IsLoading="{Binding IsBusy}"
        LoadingText="{Binding BusyMessage}"
        IndicatorType="Cupertino"
        VerticalAlignment="Center"
        HorizontalAlignment="Center"
        Margin="0 10 0 0" />

    <ListBox ItemsSource="{Binding People}">
        <ListBox.ItemTemplate>
            <DataTemplate>
                <StackPanel Margin="5">
                    <TextBlock Text="{Binding Id}" FontWeight="Bold"/>
                    <TextBlock Text="{Binding Name}" FontStyle="Italic"/>
                    <TextBlock Text="{Binding Age}" FontWeight="Bold"/>
                    <TextBlock Text="{Binding BirthDay}" FontStyle="Italic"/>
                    <TextBlock Text="{Binding Address}" FontWeight="Bold"/>
                </StackPanel>
            </DataTemplate>
        </ListBox.ItemTemplate>
    </ListBox>
</StackPanel>
</Grid>

```

